

■ LetsConnectX – Full-Stack Interview Guide

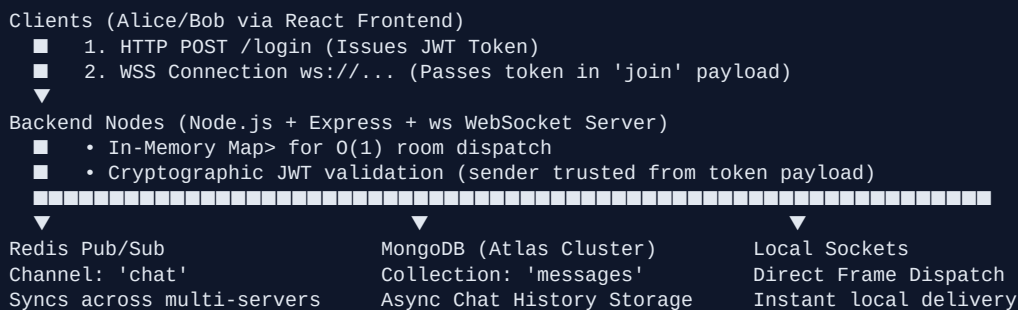
Complete System Design, WebSockets, Scaling & Engineering Q&A; Handbook

Tech Stack: React (Vite/SWC) • TypeScript • Node.js • WebSockets (ws) • Redis Pub/Sub • MongoDB • JWT Auth

1. ■ The 30-Second Elevator Pitch

"I developed **LetsConnectX**, a scalable real-time multi-room chat application designed to support distributed messaging across multiple server instances. The frontend is built with **React, TypeScript, and Tailwind CSS**, and the backend runs on **Node.js, Express, and native WebSockets (ws)**. To solve multi-server synchronization, I integrated **Redis Pub/Sub** for horizontal scaling across isolated server nodes, with **MongoDB** for message persistence and **JWT** for stateless authentication over WebSocket frames."

2. ■■ High-Level System Architecture & Design



3. ■ Core Technical Concepts & Deep Dives

Q1: Why WebSockets instead of HTTP Long Polling or Server-Sent Events (SSE)?

Technology	Protocol	Direction	Overhead	Best For
HTTP Polling	HTTP/1.1	Unidirectional (Pull)	High (New TCP+Headers)	Low-frequency data
Long Polling	HTTP/1.1	Unidirectional (Pull)	High connection churn	Legacy notifications
SSE	HTTP/2	Server → Client only	Low overhead	News/Stock tickers
WebSockets	ws:// / wss://	Full-Duplex (Bidirectional)	Extremely Low (2-6 byte frame)	Interactive Real-Time Chat

Q2: How does Horizontal Scaling work with Redis Pub/Sub?

- **The Problem:** When Alice connects to Server A and Bob connects to Server B, Server A has no socket reference to Bob. An in-memory broadcast on Server A will never reach Bob.
- **The Solution:** Server A publishes { roomId, text, sender } to the Redis chat channel. Server B (subscribed to chat) receives the message and pushes it to Bob over his local WebSocket connection.
- **Complexity:** Redis publish is **O(N)** where *N* is the number of subscribed backend servers.

Q3: How does Authentication work over WebSockets?

- **Browser Limitation:** Browser's new WebSocket(url) API does not support custom HTTP headers (e.g. Authorization: Bearer).
- **Implementation:** 1) User logs in via HTTP POST /login → receives signed JWT. 2) Client passes token inside initial join frame payload: {"type":"join","payload":{"roomId":"red","token":"..."}}. 3) Server validates with jwt.verify() and extracts trusted username from token payload.

Q4: How is Room Management implemented in memory?

- Backed by Map<string, Set<WebSocket>>. Map provides **O(1)** room lookups; Set ensures unique active sockets with **O(1)** addition and removal.
- **Memory Leak Prevention:** On socket disconnect (close event), the socket is deleted from the room Set. If users.size === 0, the room key is deleted from the Map.

Q5: How do you prevent DB queries from blocking real-time messaging?

- **Asynchronous Persistence:** Incoming messages are broadcast via WebSockets/Redis immediately. MongoDB Message.create(...) runs asynchronously in background without blocking delivery.
- **Graceful Fallback:** If MongoDB/Redis is disconnected, the server serves in-memory history cache (inMemoryHistory) and dispatches { type: 'history', payload: [] } so client never hangs on loading screen.

4. ■ End-to-End Message Lifecycle (Step-by-Step)

1. **Login:** User enters username → HTTP POST /login → Server returns signed JWT token.
2. **Connect:** Frontend opens WebSocket connection new WebSocket('wss://...').
3. **Join Room:** On open, frontend sends { type: 'join', payload: { roomId: 'red', token } }.
4. **Verify:** Backend cryptographically verifies JWT → adds socket to rooms.get('red').
5. **Load History:** Backend loads last 50 messages from MongoDB → returns { type: 'history', payload: [...] }.
6. **Send Chat:** User sends { type: 'chat', payload: { message: 'Hello', token } }.
7. **Pub/Sub:** Backend validates token → publishes { roomId, text, sender } to Redis channel.
8. **Multi-Server Broadcast:** All server instances receive Redis message → broadcast to local room sockets.
9. **UI Render:** Receiver browser receives JSON → appends to React state → auto-scrolls down.
10. **Database Save:** Backend asynchronously saves message to MongoDB Atlas.

5. ■ Scalability & Architecture at 1,000,000+ Users

Component	Current Implementation	1M+ High-Scale Architecture
Message Streaming	Redis Pub/Sub (Ephemeral)	Redis Streams / Apache Kafka (Persistent log, consumer groups, at-least-once delivery)
Load Balancing	Direct Connections	AWS ALB / Nginx with Sticky Sessions (Session Affinity) & TLS Termination
User Presence	Local Memory Set	Redis Hashes + TTL Heartbeats (SET user:id:status online EX 30)
Database Indexing	Default _id Index	Compound index on { roomId: 1, createdAt: -1 } with cursor-based pagination
Zombie Connections	OS TCP KeepAlive	WebSocket Ping/Pong protocol (heartbeat every 30s; prune dead sockets)

6. ■ Top 10 Rapid-Fire Interview Questions & Answers

1. What is the WebSocket connection handshake?

It starts as an HTTP GET request with Upgrade: websocket and Connection: Upgrade headers. The server responds with HTTP status 101 Switching Protocols to switch the persistent connection to full-duplex TCP WebSocket protocol.

2. Why use MongoDB instead of Redis for storing messages?

Redis is an in-memory datastore suited for high-speed caching and ephemeral pub/sub. MongoDB is a document database offering cost-effective persistent disk storage, secondary indexing, and rich querying for historical messages.

3. What is the difference between Redis Pub/Sub and Redis Streams?

Redis Pub/Sub is fire-and-forget (if a subscriber is offline, messages are permanently lost). Redis Streams is an append-only persistent log supporting message replay, consumer groups, and acknowledgments.

4. How do you prevent XSS (Cross-Site Scripting) attacks in the frontend?

React automatically escapes string values embedded in JSX expressions before rendering them to the DOM. On the backend, inputs are parsed as strict JSON payloads without evaluating executable script tags.

5. What is the time complexity of broadcasting to a room of M users?

Locating the room in the Map is $O(1)$ average time. Iterating through the room's Set of M active sockets and calling `socket.send()` is $O(M)$.

6. Why did you use TypeScript across frontend and backend?

End-to-end type safety eliminates runtime `TypeError`s, guarantees shared payload interfaces between client and server, and dramatically improves maintainability and refactoring confidence.

7. How does the auto-scroll feature work in React?

Using a React `useRef` attached to an empty `` at the bottom of the message container, triggered inside a `useEffect` listening to messages state: `bottomRef.current?.scrollIntoView({ behavior: 'smooth' })`.

8. What happens if a user passes custom query params like `?roomId=red`?

The React frontend parses `window.location.search` on mount and initializes the Room ID state. Configured `vercel.json` SPA rewrites guarantee that deep URLs resolve to `index.html` without 404 errors.

9. How do you handle JWT secret rotation in production?

By loading keys dynamically from secure environment variables or secret managers (AWS Secrets Manager/Vault) and validating tokens against a list of active key IDs (kid).

10. Why use Vite with SWC over traditional Webpack/Babel?

Vite uses native browser ES modules during development for instant server boot and sub-second HMR. SWC (written in Rust) compiles JSX and TypeScript 20x–50x faster than Babel.